

FastAPI Cheat Sheet

19 tables · 155 concepts · Updated 2026-07-29 · [cheatgrid.com](#)

FastAPI is a **modern, high-performance Python web framework** for building APIs based on **standard Python type hints**. It's built on top of Starlette for web handling and Pydantic for data validation, providing automatic API documentation, async support, and excellent developer experience. FastAPI excels at creating production-ready RESTful APIs quickly while maintaining type safety and performance. **One key distinction:** FastAPI's automatic validation and documentation generation from type hints eliminates boilerplate code that other frameworks require manually, making it both fast to develop with and fast to execute.

CONTENTS

Core Application Setup 1

HTTP Methods and Route Definitions 2

Request Parameters 3

Parameter Validation 3

Pydantic Models and Validation 4

Response Configuration 5

Dependency Injection 6

Error Handling 7

Authentication and Security 8

Middleware 9

Background Tasks 9

Database Integration 10

WebSockets and Real-Time Communication 10

Testing 11

Advanced Routing 12

OpenAPI Customization 12

Settings and Configuration 13

Performance Optimization 13

Advanced Patterns 15

References 15

TABLE 1

Core Application Setup

Everything starts with the `FastAPI()` instance and the path-operation decorators that hang routes off it. These are the pieces you write in your very first file — creating the app, defining an endpoint, choosing sync or async, and wiring up startup logic with a lifespan handler. The automatic `/docs` and Pydantic integration listed here are also why FastAPI feels productive from line one.

Concept	Example	Description
FastAPI app instance	<pre>app = FastAPI() app = FastAPI(title="My API", version="1.0.0")</pre>	- Creates the main application instance - accepts optional metadata like title, description, version for OpenAPI docs.
Path operation decorator	<pre>@app.get("/") @app.post("/items")</pre>	- Defines an HTTP endpoint with method and path - decorator tells FastAPI to handle requests to this route - path operations are matched in declaration order, so a fixed path like <code>/users/me</code> must be declared before a dynamic one like <code>/users/{user_id}</code> or the dynamic route shadows it.

Concept	Example	Description
Path operation function	<pre>def read_root(): return {"msg": "Hello"}</pre>	- Function executed when endpoint is hit return value is automatically serialized to JSON.
Run with FastAPI CLI	<pre>fastapi dev main.py fastapi run main.py</pre>	Recommended way to run FastAPI (included in <code>fastapi[standard]</code>) <code>dev</code> for development (auto-reload, binds localhost only); <code>run</code> for production (no reload, binds all interfaces).
Run with Uvicorn	<pre>uvicorn main:app --reload (dev) uvicorn main:app --host 0.0.0.0 --port 80 (manual/production)</pre>	- Installs and runs the ASGI server directly, useful when you want a different ASGI server or flags the <code>fastapi</code> CLI does not expose <code>fastapi dev</code> / <code>fastapi run</code> remain the recommended default, including for Docker deployments.
Async path operation	<pre>async def read_items(): return await db.fetch()</pre>	- Uses <code>async def</code> for non-blocking I/O operations, running directly on the event loop - a plain <code>def</code> function instead runs in an external threadpool, which is why blocking calls belong there, not inside <code>async def</code> .
Lifespan context manager	<pre>from contextlib import asynccontextmanager @asynccontextmanager async def lifespan(app): # startup yield # shutdown app = FastAPI(lifespan=lifespan)</pre>	- Manages startup and shutdown logic in a single function split by <code>yield</code> - replaces the deprecated <code>on_event</code> decorators; setting <code>lifespan</code> disables <code>on_event</code> entirely, it is one or the other, never both.
Automatic documentation	<pre>/docs for Swagger UI /redoc for ReDoc</pre>	- FastAPI auto-generates interactive API docs at these endpoints from your existing type hints and decorators - requires zero configuration, no separate spec file to maintain.
Pydantic integration	<pre>class Item(BaseModel): name: str price: float</pre>	- Pydantic models provide automatic validation and serialization on the request body itself - FastAPI rejects a non-matching body before your function runs, this is separate from <code>response_model</code> , which only shapes the outgoing response.

TABLE 2

HTTP Methods and Route Definitions

Each HTTP method maps to a decorator and a conventional meaning: GET reads, POST creates, PUT replaces, PATCH partially updates, DELETE removes. Following these conventions keeps your API predictable and lets caches, browsers, and other tools behave correctly. The lesser used HEAD and OPTIONS round out the set for existence checks and CORS handling.

Method	Example	Description
GET request	<pre>@app.get("/items/{id}") def read_item(id: int):</pre>	- Retrieves data no request body, typically uses path or query parameters.
POST request	<pre>@app.post("/items") def create_item(item: Item):</pre>	- Creates new resource - accepts request body defined by Pydantic model.
PUT request	<pre>@app.put("/items/{id}") def update_item(id: int, item: Item):</pre>	- Replaces entire resource - expects complete object in request body.
PATCH request	<pre>@app.patch("/items/{id}") def partial_update(id: int, item: PartialItem):</pre>	- Updates partial resource - only specified fields are modified.
DELETE request	<pre>@app.delete("/items/{id}") def delete_item(id: int):</pre>	- Removes resource - typically returns status confirmation.
HEAD request	<pre>@app.head("/items") def items_head():</pre>	- Returns headers only, no body - useful for checking resource existence or metadata.

Method	Example	Description
OPTIONS request	<pre>@app.options("/items") def items_options():</pre>	- Returns allowed HTTP methods for endpoint - often handled automatically for CORS.

TABLE 3

Request Parameters

Data reaches your endpoint from several places — the URL path, the query string, the JSON body, headers, cookies, form fields, and uploaded files — and FastAPI pulls each one in just by declaring it as a typed function argument. The standout feature is that the type annotation alone drives parsing and validation, so a path parameter declared as `int` is automatically rejected if someone passes text. Knowing which source maps to which declaration is the core skill this table builds.

Type	Example	Description
Annotated parameter syntax	<pre>from typing import Annotated q: Annotated[str None, Query(max_length=50)] = None</pre>	Recommended modern syntax (since FastAPI 0.95.0) for declaring parameters with validation - keeps type annotation and metadata together.
Path parameter	<pre>@app.get("/items/{item_id}") def read(item_id: int):</pre>	- Extracts value from URL path type validated automatically based on annotation.
Query parameter	<pre>def read_items(skip: int = 0, limit: int = 10):</pre>	- Reads from URL query string (<code>?skip=0&limit=10</code>) optional with defaults.
Request body	<pre>def create(item: Item):</pre>	- Accepts JSON payload Pydantic model validates structure and types automatically.
Multiple body parameters	<pre>def create(item: Item, user: User):</pre>	- Accepts multiple JSON objects in request body - FastAPI expects nested structure.
Header parameter	<pre>from fastapi import Header def read(user_agent: str = Header()):</pre>	- Reads HTTP headers - automatically converts your snake_case parameter name to hyphen-case to match the header.
Cookie parameter	<pre>from fastapi import Cookie def read(session_id: str = Cookie()):</pre>	- Extracts cookie values type-validated like other parameters.
Form data	<pre>from fastapi import Form def login(username: str = Form()):</pre>	- Accepts <code>application/x-www-form-urlencoded</code> - requires python-multipart installed.
File upload	<pre>from fastapi import UploadFile def upload(file: UploadFile):</pre>	- Handles file uploads <code>UploadFile</code> provides async methods and metadata like filename.
Multiple file upload	<pre>def upload(files: list[UploadFile]):</pre>	- Accepts multiple files in single request - validated as list.

TABLE 4

Parameter Validation

Beyond checking types, you often need real constraints too: a minimum length, a numeric range, a regex pattern, a required versus optional rule. The modern `Annotated` syntax shown in the first rows is the recommended way to attach these rules while keeping the type and its metadata together. Push validation this far up the stack, and your endpoint body can trust the data is already clean.

Technique	Example	Description
Annotated query validation	<pre>from typing import Annotated q: Annotated[str None, Query(min_length=3)] = None</pre>	Recommended way to use Query, Path, Header, Cookie - type annotation and validation rules stay together.
Query validation	<pre>from fastapi import Query q: str = Query(min_length=3)</pre>	Applies constraints like min/max length, regex patterns to query parameters.

Technique	Example	Description
Path validation	<pre>from fastapi import Path id: int = Path(ge=1, le=1000)</pre>	Validates path parameters with numeric constraints like ge (greater/equal), le (less/equal).
Required query parameter	<pre>q: Annotated[str, Query(min_length=1)]</pre>	- Modern approach: no default = required when using Annotated - older approach: <code>q: str = Query(...)</code> with ellipsis.
Optional with None	<pre>q: str None = None</pre>	- Python 3.10+ union syntax for optional parameters - older versions use <code>Optional[str]</code> .
List query parameter	<pre>tags: list[str] = Query([])</pre>	- Accepts multiple values for same parameter (<code>?tags=a&tags=b</code>) - validated as list.
Alias parameter	<pre>item_query: str = Query(alias="item-query")</pre>	Maps hyphenated URL param to <code>snake_case</code> Python variable.
Deprecated parameter	<pre>old_param: str = Query(deprecated=True)</pre>	- Marks parameter as deprecated in OpenAPI docs - still functional but discouraged.
Regex validation	<pre>code: str = Query(pattern="^[A-Z]{3}\$")</pre>	Validates string parameter against regex pattern .
Numeric constraints	<pre>price: float = Query(gt=0, le=9999.99)</pre>	Applies gt (greater than), lt (less than), ge , le constraints to numbers.

TABLE 5

Pydantic Models and Validation

Pydantic models are the heart of how FastAPI validates and serializes structured data, a `BaseModel` subclass defines the shape of a request or response, and FastAPI does the rest. This table covers the tools for going beyond simple fields: per-field constraints, custom field and model validators for logic that types can't express, computed fields, and model inheritance for reuse. Note this is Pydantic v2, so `model_config` and `field_validator` replace the older patterns you may have seen.

Concept	Example	Description
BaseModel	<pre>class Item(BaseModel): name: str price: float</pre>	- Base class for all Pydantic models - provides automatic validation and serialization.
Field constraints	<pre>from pydantic import Field price: float = Field(gt=0)</pre>	- Adds validation rules directly to model fields - similar to Query/Path but for models.
Field description	<pre>name: str = Field(description="Item name")</pre>	- Adds documentation to field - appears in OpenAPI schema.
Field example	<pre>price: float = Field(examples=[9.99, 19.99])</pre>	Provides example values for API documentation and testing.
Field validator	<pre>@field_validator('email') def check_email(cls, v):</pre>	- Custom validation logic for specific field - runs after type validation.
Model validator	<pre>@model_validator(mode='after') def check_passwords(self):</pre>	- Validates entire model after individual fields - useful for cross-field validation.
model_config	<pre>from pydantic import ConfigDict model_config = ConfigDict(str_strip_whitespace=True)</pre>	- Configures model behavior using Pydantic v2's <code>ConfigDict</code> - replaces the old inner <code>class Config</code> .
Optional fields	<pre>description: str None = None</pre>	- Field can be omitted or null - uses Python union type or <code>Optional</code> .
Computed field	<pre>@computed_field @property def total(self) -> float:</pre>	- Adds calculated field to model - included in serialization but not validation.

Concept	Example	Description
Model inheritance	<pre>class ExtendedItem(Item): category: str</pre>	- Inherits fields from base model - enables model reuse and extension.

TABLE 6

Response Configuration

Controlling what goes back to the client is just as important as parsing what comes in. The `response_model` and return-type annotation filter and validate your output, handy for stripping a password field before it ever leaves the server, while the various response classes let you return HTML, files, redirects, or streamed data instead of plain JSON. Status codes, custom headers, and cookies round out the toolkit for shaping a precise response.

Technique	Example	Description
Response model	<pre>@app.post("/items", response_model=Item)</pre>	- Specifies return type for validation and docs - filters response to match model.
Return type annotation	<pre>async def create_item(item: Item) -> Item:</pre>	- Modern preferred approach for response type declaration - FastAPI uses the return annotation for validation, filtering, and docs.
Response model exclude	<pre>response_model=User, response_model_exclude={"password"}</pre>	Removes specified fields from the serialized response only - the field is still fetched and held in memory first, so pair it with a separate output model for real secrets.
Status code	<pre>@app.post("/items", status_code=201)</pre>	- Sets HTTP status code for the response - FastAPI's default is 200 for every method, 201 for POST is only a convention you set explicitly.
JSONResponse	<pre>from fastapi.responses import JSONResponse return JSONResponse(content=data)</pre>	Explicit JSON response with custom headers or status code.
HTMLResponse	<pre>from fastapi.responses import HTMLResponse return HTMLResponse(content=html)</pre>	- Returns HTML content - sets Content-Type to text/html.
RedirectResponse	<pre>from fastapi.responses import RedirectResponse return RedirectResponse(url="/new")</pre>	HTTP redirect to different URL - status_code defaults to 307.
FileResponse	<pre>from fastapi.responses import FileResponse return FileResponse(path="file.pdf")</pre>	- Serves static files - handles content-type and download headers automatically.
StreamingResponse	<pre>from fastapi.responses import StreamingResponse return StreamingResponse(generator)</pre>	- Streams large data in chunks - memory-efficient for big responses.
ORJSONResponse	<pre>from fastapi.responses import ORJSONResponse return ORJSONResponse(content=data)</pre>	- Uses the third party orjson library for JSON serialization - a faster alternative response class, not a difference in what data it returns.
Response headers	<pre>from fastapi import Response response.headers["X-Custom"] = "value"</pre>	Sets custom HTTP headers in response.
Set cookies	<pre>response.set_cookie(key="session", value="abc")</pre>	- Adds cookie to response - supports httponly, secure, samesite options.

TABLE 7

Dependency Injection

Dependency injection is one of FastAPI's most loved features — `Depends()` lets you factor out reusable logic like database sessions, authentication, and pagination, then declare it as a parameter wherever you need it. Dependencies can nest, run as `yield` generators that clean up after the response, apply at the route, router, or app level, and be swapped out entirely for testing. Lean on this pattern and your endpoints stay thin while shared concerns live in one place.

Pattern	Example	Description
Basic dependency	<pre>from typing import Annotated def route(db: Annotated[DB, Depends(get_db)]):</pre>	- Function providing reusable logic - recommended Annotated syntax (since 0.95.0) keeps type and dependency together.
Annotated type alias dependency	<pre>DBDep = Annotated[Session, Depends(get_db)] def endpoint(db: DBDep):</pre>	- Defines reusable type alias for common dependency - reduces repetition across many routes.
Async dependency	<pre>async def get_session(): async with Session() as s: yield s</pre>	Async generator dependency <code>sync (def)</code> and <code>async (async def)</code> dependencies can be freely mixed in the same app, FastAPI runs each correctly.
Class dependency	<pre>class Pagination: def __init__(self, skip=0):</pre>	- Class instances as dependencies <code>__init__</code> parameters become sub-dependencies, validated the same way as path operation parameters, and give the editor type completion a dict cannot.
Nested dependencies	<pre>def get_user(token = Depends(get_token)):</pre>	- Dependency can depend on other dependencies - creates dependency chain; a sub-dependency shared by several dependants in one request still runs only once per request.
Global dependencies	<pre>app = FastAPI(dependencies=[D epends(verify)])</pre>	- Applied to all routes in application - useful for auth, logging; combines with router level and route level dependencies rather than overriding them.
Router dependencies	<pre>router = APIRouter(dependenci es=[Depends(check)])</pre>	- Applied to all routes in router - scoped to specific module; when a route also has its own dependencies, FastAPI resolves the router's first, then the path decorator's, then normal parameter dependencies.
Yield dependency	<pre>def get_db(): db = Database() yield db db.close()</pre>	- Provides resource cleanup after the response is already sent, not before - if the code after <code>yield</code> catches an exception without re-raising it, FastAPI never learns the exception happened.
Dependency override	<pre>app.dependency_overrides[get_ db] = mock_db</pre>	- Replaces dependency for testing - a plain dict swap, no route code changes - stays active until you manually reset it with <code>app.dependency_overrides = {}</code>.
Dependencies in path decorator	<pre>@app.get("/items", dependenci es=[Depends(verify)])</pre>	- Dependency executed but result not used in function - runs unconditionally for its side effects only, such as raising an error on an invalid header.

TABLE 8

Error Handling

When something goes wrong, you want a clean, consistent error response rather than a raw stack trace. Raising `HTTPException` is the everyday move for returning a 404 or 403, while custom exceptions paired with exception handlers let you centralize how whole categories of errors turn into responses. You can even override FastAPI's default validation-error format when you need errors shaped a particular way.

Approach	Example	Description
HTTPException	<pre>from fastapi import HTTPException raise HTTPException(status_code=404, detail="Not found")</pre>	• Built-in exception for HTTP errors • caught by a default handler and formatted as JSON automatically, no setup needed.
Custom exception	<pre>class CustomError(Exception): pass</pre>	• Define a custom exception class by subclassing plain <code>Exception</code> • does nothing on its own until paired with a handler.
Exception handler	<pre>@app.exception_handler(CustomError) def handler(request, exc):</pre>	• Catches a specific exception type globally, and any subclass of it too • returns a custom response.
Validation error handler	<pre>from fastapi.exceptions import RequestValidationError @app.exception_handler(RequestValidatio nError)</pre>	• Customizes Pydantic's 422 validation error response format • needs its own registration, overriding <code>HTTPException</code>'s handler alone does not affect it.
Status code in exception	<pre>raise HTTPException(status_code=403, detail="Forbidden")</pre>	• Sets the HTTP status code directly when raising the exception • common codes: 400, 401, 403, 404, 500.
Custom headers in exception	<pre>raise HTTPException(status_code=401, headers={"WWW-Authenticate": "Bearer"})</pre>	• Adds custom headers to an error response • useful for auth challenges, carries real protocol meaning, not just decoration.
Override default handlers	<pre>from starlette.exceptions import HTTPException as StarletteHTTPException @app.exception_handler(StarletteHTTPExc eption)</pre>	• Replaces FastAPI's default error formatting • registers on Starlette's <code>HTTPException</code> so it also catches errors Starlette raises internally, not just FastAPI's own subclass.

Topic edits

- Description: kept-as-is (already specific and learning-focused, correctly frames the progression from `HTTPException` to custom handlers to overriding validation formatting)
- Rows added: none
- Rows removed: none
- Rows reordered: no
- Column 1 link updates: none
- Row description rewrites: `HTTPException` (clarified it is caught by a default handler automatically, no setup needed, to head off confusion with unhandled exceptions), Custom exception (clarified it does nothing on its own until paired with a handler), Exception handler (clarified it also catches subclasses of the registered type), Validation error handler (clarified overriding the `HTTPException` handler alone does not affect it), Custom headers in exception (clarified the header carries real protocol meaning, not just decoration), Override default handlers (clarified it must register on Starlette's `HTTPException` specifically to also catch Starlette's own internally raised errors)
- Row example rewrites: none (all examples already match the official docs' canonical syntax)
- Duplicates merged: none
- Deprecated demotions: none

TABLE 9

Authentication and Security

FastAPI ships with security utilities that plug straight into its dependency system and its OpenAPI docs. The rows below walk through the common building blocks: OAuth2 password flows, API keys sent via headers, query parameters, or cookies, JWT creation and verification with PyJWT, and password hashing with pwdlib, and how they combine into a `get_current_user` dependency you reuse across protected routes. Security scopes at the end add fine-grained, permission-based access on top.

Mechanism	Example	Description
OAuth2 password flow	<pre>from fastapi.security import OAuth2PasswordBearer oauth2 = OAuth2PasswordBearer(tokenUrl="token")</pre>	• Standard OAuth2 flow for password-based authentication • extracts the bearer token string from the header only, your own dependency must still verify it.
OAuth2 password request	<pre>from fastapi.security import OAuth2PasswordRequestForm def login(form: OAuth2PasswordRequestForm = Depends()):</pre>	• Form data model for username/password login • must be sent as form fields, not JSON, the field names are fixed by the OAuth2 spec.
API key in header	<pre>from fastapi.security import APIKeyHeader api_key = APIKeyHeader(name="X-API-Key")</pre>	• Extracts API key from a custom header • only reads the value, your code must still check that it is a valid key.
API key in query	<pre>from fastapi.security import APIKeyQuery api_key = APIKeyQuery(name="api_key")</pre>	• Reads API key from a query parameter • less secure than a header, query strings often end up in server logs and browser history.
API key in cookie	<pre>from fastapi.security import APIKeyCookie api_key = APIKeyCookie(name="session")</pre>	• Extracts API key from a cookie • cookies are attached to requests automatically by the browser, which is what makes cookie-based auth CSRF-prone without extra defenses.
JWT token creation	<pre>import jwt token = jwt.encode(data, SECRET, algorithm="HS256")</pre>	• Encodes a JWT token with a payload and a secret key • FastAPI's current docs use the actively maintained PyJWT library (<code>import jwt</code>), not the older python-jose.
JWT token verification	<pre>import jwt from jwt.exceptions import InvalidTokenError payload = jwt.decode(token, SECRET, algorithms=["HS256"])</pre>	• Decodes and verifies a JWT's signature • raises <code>InvalidTokenError</code> if invalid, the payload itself is only base64-encoded, never encrypted.
Password hashing	<pre>from pwdlib import PasswordHash password_hash = PasswordHash.recommended()</pre>	• Hashes passwords for storage, one-way, never store plain passwords • FastAPI's current docs recommend pwdlib with Argon2, bcrypt remains supported for compatibility with older hashes.
Current user dependency	<pre>async def get_current_user(token = Depends(oauth2)): # decode token, look up user, check active</pre>	• Dependency that decodes the token and looks up the user • also checks the account is still active, a valid signature alone is not enough.
Security scopes	<pre>@app.get("/items", dependencies=[Depen ds(has_scope("read:items"))])</pre>	• Permission-based access control using OAuth2 scopes • your own dependency must compare the token's scopes against the required ones and raise the error itself.

TABLE 10

Middleware

Middleware wraps every request and response, making it the right place for cross-cutting concerns that shouldn't live in individual endpoints. CORS is the one nearly everyone needs first, while the others handle compression, HTTPS enforcement, trusted-host checks, and sessions — plus the option to write your own function- or class-based middleware for custom logic like timing or request logging.

Type	Example	Description
CORS middleware	<pre>from fastapi.middleware.cors import CORSMiddleware app.add_middleware(CORSMiddleware, allow_origins=["https://example.com"], allow_credentials=True)</pre>	- Enables Cross-Origin Resource Sharing - configures which domains can access API.
Custom function middleware	<pre>@app.middleware("http") async def add_header(request, call_next):</pre>	Function-based middleware - runs before and after every request.
Custom class middleware	<pre>class TimingMiddleware(BaseHTTPMiddleware): async def dispatch(self, request, call_next):</pre>	Class-based middleware - supports configuration via <code>__init__</code> for more complex logic.
Trusted host middleware	<pre>from starlette.middleware.trustedhost import TrustedHostMiddleware app.add_middleware(TrustedHostMiddleware, allowed_hosts=["*.example.com"])</pre>	- Validates Host header - prevents HTTP Host header attacks.
HTTPS redirect middleware	<pre>from starlette.middleware.httpsredirect import HTTPSRedirectMiddleware app.add_middleware(HTTPSRedirectMiddleware)</pre>	Redirects HTTP to HTTPS - enforces secure connections.
GZip middleware	<pre>from fastapi.middleware.gzip import GZipMiddleware app.add_middleware(GZipMiddleware, minimum_size=1000)</pre>	Compresses responses over specified size - reduces bandwidth.
Session middleware	<pre>from starlette.middleware.sessions import SessionMiddleware app.add_middleware(SessionMiddleware, secret_key="key")</pre>	Adds session support via signed cookies.

TABLE 11

Background Tasks

Some work, like sending a confirmation email or writing a log line, shouldn't make the user wait for the response. FastAPI's built-in BackgroundTasks handles these lightweight cases right inside the same process, with no extra infrastructure required. For heavier or long-running jobs that need their own workers, this table also covers Celery and ARQ as the next step up.

Approach	Example	Description
BackgroundTasks	<pre>from fastapi import BackgroundTasks def task(email: str): send_email(email)</pre>	Simple background function executed after response - no external queue needed.
Add background task	<pre>def endpoint(bg: BackgroundTasks): bg.add_task(send_email, to="user@example.com")</pre>	- Schedules function to run after response sent - tasks run in same process.
Multiple background tasks	<pre>bg.add_task(task1) bg.add_task(task2)</pre>	Multiple tasks run sequentially in order added - an exception in one stops the rest from running
Background task with dependencies	<pre>def get_query(background_tasks: BackgroundTasks, q: str = None): background_tasks.add_task(write_log, message)</pre>	- A dependency can receive <code>BackgroundTasks</code> and call <code>.add_task()</code> itself - its data is resolved during the request, before the task runs, not by <code>Depends()</code> on the task function itself.

Approach	Example	Description
Celery integration	<pre>from celery import Celery celery = Celery("tasks", broker="redis://localhost") @celery.task def heavy_task():</pre>	• Distributed task queue for heavy/long-running jobs • runs in separate workers outside request cycle.
ARQ for async tasks	<pre>from arq import create_pool redis = await create_pool()</pre>	• Redis-based async task queue built on asyncio • alternative to Celery for async-first applications.

TABLE 12

Database Integration

Most FastAPI apps talk to a database through SQLAlchemy, and the choice between async and sync sessions sets the tone for everything else. These rows cover defining ORM models, running queries the SQLAlchemy 2.0 way, injecting a session as a dependency so it closes cleanly after each request, and managing schema changes with Alembic. Connection pooling at the end is the detail that keeps a busy API from exhausting the database.

Technique	Example	Description
SQLAlchemy async session	<pre>from sqlalchemy.ext.asyncio import AsyncSession async def get_db(): yield session</pre>	• Async database session for non-blocking DB operations • requires an asyncio-native driver like asyncpg or aiosqlite.
SQLAlchemy sync session	<pre>from sqlalchemy.orm import Session def get_db(): yield db</pre>	• Synchronous session • simpler but blocks during queries.
Database model	<pre>from sqlalchemy.orm import Mapped, mapped_column class User(Base): id: Mapped[int] = mapped_column(primary_key=True)</pre>	• ORM model defining table structure via typed columns • the modern <code>Mapped / mapped_column()</code> style replaces the older <code>Column(...)</code> form.
Async database query	<pre>result = await session.execute(select(User)) users = result.scalars().all()</pre>	• Executes async query • uses SQLAlchemy 2.0 style.
Database dependency	<pre>def endpoint(db: Session = Depends(get_db)):</pre>	• Injects database session into route • session closed automatically after request via the dependency's <code>yield / finally</code> .
Alembic migrations	<pre>alembic init alembic alembic revision --autogenerate alembic upgrade head</pre>	• Database schema migrations • tracks and applies schema changes over time; always review autogenerated migrations before applying, since they can miss renames.
Connection pooling	<pre>engine = create_async_engine(url, pool_size=20, max_overflow=10)</pre>	• Reuses database connections instead of opening a new one per query • critical for production performance.

TABLE 13

WebSockets and Real-Time Communication

When you need a live, two-way channel, like chat, notifications, or a streaming dashboard, a plain request/response cycle won't do. FastAPI's WebSocket support gives you the full lifecycle: accept the connection, receive and send text or JSON in a loop, and handle the disconnect cleanly. The final SSE row is the simpler choice when updates only need to flow one way, from server to client.

Feature	Example	Description
WebSocket endpoint	<pre>from fastapi import WebSocket @app.websocket("/ws") async def websocket_endpoint(websocket: WebSocket):</pre>	- Defines WebSocket route - full-duplex communication channel.
Accept connection	<pre>await websocket.accept()</pre>	Establishes WebSocket connection - must be called before send/receive.
Receive message	<pre>data = await websocket.receive_text() data = await websocket.receive_json()</pre>	Reads message from client - supports text, JSON, or bytes.
Send message	<pre>await websocket.send_text("Hello") await websocket.send_json({"msg": "Hi"})</pre>	Sends message to client.
WebSocket disconnect	<pre>try: while True: await websocket.receive_text() except WebSocketDisconnect:</pre>	- Handles client disconnect - exception raised when connection closes.
Server-Sent Events	<pre>from fastapi.responses import StreamingResponse def event_stream(): yield f"data: {msg}\n\n"</pre>	One-way streaming from server to client - simpler than WebSockets for uni-directional updates.

TABLE 14

Testing

FastAPI makes testing genuinely pleasant because `TestClient` runs your app in-process — no server to spin up. These rows show the standard pytest workflow, the context-manager form that triggers lifespan events, the httpx-based async client for testing async code properly, and the dependency-override trick that swaps a real database for a mock without touching application code.

Tool	Example	Description
TestClient	<pre>from fastapi.testclient import TestClient client = TestClient(app) response = client.get("/")</pre>	Synchronous test client - tests FastAPI without running server.
Test with pytest	<pre>def test_read_main(): response = client.get("/") assert response.status_code == 200</pre>	- Standard pytest patterns work with TestClient - uses fixtures for setup.
TestClient as context manager	<pre>with TestClient(app) as client: response = client.get("/")</pre>	Triggers lifespan events during testing - startup/shutdown run automatically.
Async test client	<pre>from httpx import ASGITransport, AsyncClient async with AsyncClient(transport=ASGITransport(app=app), base_url="http://test") as client: response = await client.get("/")</pre>	Async testing with httpx - lets a test function <code>await</code> other async code, like async database calls.
Override dependency	<pre>app.dependency_overrides[get_db] = lambda: mock_db</pre>	Replaces real dependency with mock for testing - no code changes needed.
Test file upload	<pre>files = {"file": ("test.txt", b"content", "text/plain")} client.post("/upload", files=files)</pre>	Simulates file upload in tests - tuple format: (filename, content, content-type).
Test with pytest fixtures	<pre>@pytest.fixture def client(): return TestClient(app)</pre>	Reusable test setup - pytest fixture creates client for all tests.

TABLE 15

Advanced Routing

As an app grows past a single file, `APIRouter` is how you split routes into focused modules with shared prefixes, tags, and dependencies, then mount them onto the main app with `include_router()`. Sub-application mounting and static file serving are a different tool entirely, since they hand a path over to a genuinely separate ASGI app instead of just organizing routes. The route-priority row captures a subtle gotcha: declare `/users/me` before `/users/{id}`, or the parameterized route swallows it silently.

Concept	Example	Description
APIRouter	<pre>from fastapi import APIRouter router = APIRouter(prefix="/items", tags=["items"])</pre>	• Modular routing • groups related endpoints with shared prefix and tags.
Include router	<pre>app.include_router(router)</pre>	• Mounts router to main app • enables splitting large apps into modules.
Sub-application mounting	<pre>app.mount("/v1", app_v1)</pre>	• Mounts independent FastAPI app at path • useful for API versioning.
Static files	<pre>from fastapi.staticfiles import StaticFiles app.mount("/static", StaticFiles(directory="static"))</pre>	Serves static files from directory.
Router dependencies	<pre>router = APIRouter(dependencies=[Depends(verify_token)])</pre>	• Applies dependency to all routes in router • cleaner than per-route.
Router tags	<pre>router = APIRouter(tags=["items"])</pre>	• Groups endpoints in OpenAPI docs • creates collapsible sections.
Route priority	Define more specific routes first : <pre>@app.get("/users/me") @app.get("/users/{id}")</pre>	• FastAPI matches first matching route • specific paths before parameterized ones.

TABLE 16

OpenAPI Customization

FastAPI generates an OpenAPI schema and interactive docs for free, but real projects want to shape them. These rows let you set titles and descriptions, group endpoints under documented tags, add example responses, deprecate old routes, change or disable the docs URLs, and tweak Swagger UI behavior. Polishing the generated docs is what turns a working API into one other developers actually enjoy consuming.

Element	Example	Description
Metadata	<pre>app = FastAPI(title="My API", description="Docs", version="1.0")</pre>	Sets API title, description, version in OpenAPI schema and docs.
Tags metadata	<pre>tags_metadata = [{"name": "items", "description": "Item operations"}] app = FastAPI(openapi_tags=tags_metadata)</pre>	Adds descriptions to tag groups in documentation.
Custom OpenAPI schema	<pre>def custom_openapi(): # modify schema app.openapi = custom_openapi</pre>	• Overrides OpenAPI generation • add custom extensions or modify schema • caches the result in <code>app.openapi_schema</code> so it is only built once.
Disable docs	<pre>app = FastAPI(docs_url=None, redoc_url=None)</pre>	• Hides the Swagger UI and ReDoc pages • the <code>/openapi.json</code> schema stays public by default, set <code>openapi_url=None</code> too if it needs to be gone.
Custom docs URL	<pre>app = FastAPI(docs_url="/api/docs")</pre>	Changes Swagger UI path from default <code>/docs</code> .
Operation ID	<pre>@app.get("/items", operation_id="get_all_items")</pre>	• Sets unique operation identifier in OpenAPI • useful for code generation tools.

Element	Example	Description
Deprecate endpoint	<code>@app.get("/old", deprecated=True)</code>	- Marks endpoint as deprecated in docs - still functional.
Response examples	<code>responses={200: {"description": "OK", "content": {"application/json": {"example": {...}}}}</code>	Adds example responses to OpenAPI docs.
Separate input/output schemas	<code>app = FastAPI(separate_input_output_schemas=False)</code>	- FastAPI 0.102.0+ generates separate OpenAPI schemas for input and output with Pydantic v2 - set <code>False</code> to disable.
Configure Swagger UI	<code>app = FastAPI(swagger_ui_parameters={"defaultModelsExpandDepth": -1})</code>	Customizes Swagger UI behavior like theme, syntax highlighting, model expansion.

TABLE 17

Settings and Configuration

Pydantic Settings brings the same type-safe validation you use for requests to your application config, reading values from environment variables and `.env` files and failing fast at startup if something's missing. Injecting settings as a dependency keeps them easy to mock in tests, and caching them with `lru_cache` means the config is parsed once. This is Pydantic Settings v2, so `SettingsConfigDict` replaces the old inner `Config` class.

Technique	Example	Description
Pydantic Settings	<code>from pydantic_settings import BaseSettings class Settings(BaseSettings): api_key: str</code>	Type-safe configuration from environment variables - validates config at startup.
Read .env file	<code>from pydantic_settings import SettingsConfigDict model_config = SettingsConfigDict(env_file=".env")</code>	- Loads environment variables from .env using Pydantic Settings v2 - replaces old inner class <code>Config: env_file = ".env"</code> pattern.
Settings dependency	<code>def get_settings(): return Settings() settings: Settings = Depends(get_settings)</code>	Injects settings into routes - enables easy mocking in tests.
Cached settings	<code>@lru_cache def get_settings(): return Settings()</code>	Loads settings once and caches - improves performance.
Multiple environments	<code>class Settings(BaseSettings): env: str = "dev" database_url: str</code>	- Uses different config per environment - set via ENV variable.

TABLE 18

Performance Optimization

FastAPI is already fast, but these levers squeeze out more under real load. Writing I/O-bound handlers as `async` unlocks concurrency, Redis caching and HTTP cache headers cut repeated work, connection pooling protects the database, and ORJSON speeds up serialization. The deployment rows — Uvicorn workers, Gunicorn, and the Docker pattern — cover putting all of that into production across multiple CPU cores.

Strategy	Example	Description
Async operations	<code>async def fetch_data(): return await http_client.get(url)</code>	- Use <code>async def</code> for I/O-bound operations - enables concurrent request handling, but a CPU-heavy <code>async def</code> with no real <code>await</code> still blocks the whole event loop.
Redis caching	<code>FastAPIRedis(app).lifespan().cach hing() @app.get("/items/{id}", dependen cies=[Depends(cache(ttl=60))])</code>	Caches responses in Redis to reduce database load for repeated queries - needs an explicit eviction step, or users can see stale data until the TTL expires.

Strategy	Example	Description
Response caching headers	<pre>def get_items(response: Response): response.headers["Cache-Control"] = "max-age=3600"</pre>	• Sets HTTP caching headers so clients, proxies, and CDNs know how long to reuse a response • purely advisory, FastAPI still runs the route on every request unless an actual cache sits in front of it.
Connection pooling	<pre>engine = create_engine(url, pool_size=20)</pre>	• Reuses database connections instead of opening a new one per request • an undersized pool queues up requests and raises timeout errors once concurrent load exceeds its capacity.
ORJSONResponse	<pre>app = FastAPI(default_response_class=ORJSONResponse)</pre>	• Uses the Rust based orjson library to serialize responses faster than the standard <code>json</code> module • a <code>response_model</code> can already reach similar speed, and <code>orjson</code> has its own edge cases like turning NaN into <code>null</code> .
Uvicorn workers	<pre>uvicorn main:app --workers 4</pre>	• Runs multiple worker processes to use multiple CPU cores • each worker is a separate process with its own memory, so in-memory caches or rate limiters aren't shared across workers.
Gunicorn + Uvicorn	<pre>gunicorn main:app --workers 4 --worker-class uvicorn.workers.UvicornWorker</pre>	• Process manager that starts and supervises Uvicorn worker processes, restarting any that crash • a common production deployment combination.
Docker deployment	<pre>FROM python:3.14 RUN pip install fastapi[standard] CMD ["fastapi", "run", "main.py", "--port", "80"]</pre>	• Official Docker image pattern for FastAPI production deployment • on Kubernetes or similar clusters, run a single process per container and let the cluster handle replication instead of adding <code>--workers</code> too.

Topic edits

- Description: kept-as-is (already specific and learning-focused, names the concrete levers covered by the table)
- Rows added: none
- Rows removed: none
- Rows reordered: no
- Column 1 link updates: Redis caching: <https://medium.com/@kirant3ja/redis-hybrid-caching-in-fastapi-af0bf0ec40cb> → <https://redis.io/docs/latest/integrate/fastapi/> (official Redis docs for the FastAPI integration, replacing a third-party Medium post); Response caching headers: <https://mojoauth.com/implement-caching/implement-caching-with-fastapi> → <https://fastapi.tiangolo.com/advanced/response-headers/> (official FastAPI docs, replacing a low-authority blog whose own example used an invalid FastAPI syntax); Uvicorn workers: <https://www.uvicorn.org/deployment/> → <https://fastapi.tiangolo.com/deployment/server-workers/> (the old uvicorn.org domain no longer resolves; linking FastAPI's own official Uvicorn-workers doc instead); Gunicorn + Uvicorn: <https://www.uvicorn.org/deployment/> → <https://uvicorn.dev/deployment/> (uvicorn.org is dead, the project's docs now live at uvicorn.dev)
- Row description rewrites: Async operations (added the CPU-bound blocking caveat), Redis caching (added the eviction/staleness caveat), Response caching headers (clarified the header is advisory, not enforced server-side, and fixed the accompanying example), Connection pooling (added the undersized-pool production-timeout consequence), ORJSONResponse (added the response_model-parity and NaN-handling nuance), Uvicorn workers (added the per-process memory isolation caveat), Docker deployment (added the one-process-per-container guidance for clusters)
- Row example rewrites: Response caching headers (the prior example, `@app.get("/", response_headers={...})` , is not valid FastAPI syntax; there is no `response_headers` parameter on a path operation decorator, replaced with the documented `Response` parameter pattern); Redis caching (replaced the third-party `fastapi_cache` snippet with the official `fastapi-redis-sdk` pattern matching the new Column 1 link)
- Duplicates merged: none
- Deprecated demotions: none

TABLE 19

Advanced Patterns

These are the extras you reach for once the basics are in place and the app is heading toward production. Rate limiting protects you from abuse, API versioning lets the interface evolve without breaking clients, and a GraphQL endpoint or class-based views fit specialized needs. Request-ID tracking and alternative serialization like MessagePack round out a toolkit for APIs that need to scale and be observable.

Pattern	Example	Description
GraphQL integration	<pre>from strawberry.fastapi import GraphQLRouter app.include_router(GraphQLRouter(schema), prefix="/graphql")</pre>	• Adds GraphQL endpoint using Strawberry • enables flexible query language.
Rate limiting	<pre>from slowapi import Limiter @limiter.limit("5/minute") def endpoint():</pre>	• Throttles requests per IP/user • prevents API abuse.
API versioning (URL)	<pre>app.mount("/v1", v1_app) app.mount("/v2", v2_app)</pre>	• Versions API via URL path • most common versioning strategy.
API versioning (header)	<pre>version = request.headers.get("API-Version", "v1")</pre>	• Reads version from custom header • cleaner URLs but less discoverable.
Class-based views	<pre>from fastapi_utils.cbv import cbv @cbv(router) class ItemAPI: @router.get("/items")</pre>	• Groups related endpoints in class • reduces boilerplate for CRUD operations.
MessagePack serialization	<pre>from msgpack_asgi import MessagePackMiddleware app.add_middleware(MessagePackMiddleware)</pre>	• Uses binary format for smaller payloads • faster than JSON for large data.
Request ID tracking	<pre>@app.middleware("http") async def add_request_id(request, call_next):</pre>	• Adds unique ID to each request • enables distributed tracing.

References

FastAPI & Python web ecosystem

1. Alembic migrations - <https://alembic.sqlalchemy.org/en/latest/autogenerate.html>
2. Annotated parameter syntax - <https://fastapi.tiangolo.com/tutorial/query-params-str-validations/>
3. API key in header - <https://fastapi.tiangolo.com/reference/security/>
4. Async path operation - <https://fastapi.tiangolo.com/async/>
5. Async test client - <https://fastapi.tiangolo.com/advanced/async-tests/>
6. BackgroundTasks - <https://fastapi.tiangolo.com/tutorial/background-tasks/>
7. BaseModel - <https://docs.pydantic.dev/latest/concepts/models/>
8. Basic dependency - <https://fastapi.tiangolo.com/tutorial/dependencies/>
9. Class dependency - <https://fastapi.tiangolo.com/tutorial/dependencies/classes-as-dependencies/>
10. Computed field - <https://docs.pydantic.dev/latest/concepts/fields/#the-computed-field-decorator>
11. Configure Swagger UI - <https://fastapi.tiangolo.com/how-to/configure-swagger-ui/>
12. Connection pooling - <https://docs.sqlalchemy.org/en/20/core/pooling.html>
13. Cookie parameter - <https://fastapi.tiangolo.com/tutorial/cookie-params/>
14. Custom class middleware - <https://www.starlette.io/middleware/>
15. Custom function middleware - <https://fastapi.tiangolo.com/tutorial/middleware/>
16. Custom OpenAPI schema - <https://fastapi.tiangolo.com/how-to/extending-openapi/>
17. Database model - https://docs.sqlalchemy.org/en/20/orm/declarative_tables.html
18. DELETE request - <https://fastapi.tiangolo.com/reference/fastapi/>
19. Dependencies in path decorator - <https://fastapi.tiangolo.com/tutorial/dependencies/dependencies-in-path-operation-decorators/>
20. Dependency override - <https://fastapi.tiangolo.com/advanced/testing-dependencies/>
21. Docker deployment - <https://fastapi.tiangolo.com/deployment/docker/>
22. FastAPI app instance - <https://fastapi.tiangolo.com/tutorial/first-steps/>
23. Field constraints - <https://docs.pydantic.dev/latest/concepts/fields/>
24. Field validator - <https://docs.pydantic.dev/latest/concepts/validators/>
25. File upload - <https://fastapi.tiangolo.com/tutorial/request-files/>
26. Form data - <https://fastapi.tiangolo.com/tutorial/request-forms/>
27. Global dependencies - <https://fastapi.tiangolo.com/tutorial/dependencies/global-dependencies/>
28. Header parameter - <https://fastapi.tiangolo.com/tutorial/header-params/>
29. HTTPException - <https://fastapi.tiangolo.com/tutorial/handling-errors/>
30. JSONResponse - <https://fastapi.tiangolo.com/advanced/custom-response/>
31. Lifespan context manager - <https://fastapi.tiangolo.com/advanced/events/>
32. Metadata - <https://fastapi.tiangolo.com/tutorial/metadata/>
33. model_config - <https://docs.pydantic.dev/latest/concepts/config/>
34. Multiple body parameters - <https://fastapi.tiangolo.com/tutorial/body-multiple-params/>

- 35. Multiple environments - https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- 37. OAuth2 password flow - <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>
- 39. Operation ID - <https://fastapi.tiangolo.com/advanced/path-operation-advanced-configuration/>
- 41. Path parameter - <https://fastapi.tiangolo.com/tutorial/path-params/>
- 43. PUT request - <https://fastapi.tiangolo.com/tutorial/body-updates/>
- 45. Pydantic Settings - <https://fastapi.tiangolo.com/advanced/settings/>
- 47. Response examples - <https://fastapi.tiangolo.com/advanced/additional-responses/>
- 49. Response model - <https://fastapi.tiangolo.com/tutorial/response-model/>
- 51. Router tags - <https://fastapi.tiangolo.com/tutorial/path-operation-configuration/>
- 53. Run with Uvicorn - <https://fastapi.tiangolo.com/deployment/manually/>
- 55. Separate input/output schemas - <https://fastapi.tiangolo.com/how-to/separate-openapi-schemas/>
- 57. SQLAlchemy async session - <https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html>
- 59. Static files - <https://fastapi.tiangolo.com/tutorial/static-files/>
- 61. Sub-application mounting - <https://fastapi.tiangolo.com/advanced/sub-applications/>
- 63. TestClient as context manager - <https://fastapi.tiangolo.com/advanced/testing-events/>
- 65. Uvicorn workers - <https://fastapi.tiangolo.com/deployment/server-workers/>
- 67. Yield dependency - <https://fastapi.tiangolo.com/tutorial/dependencies/dependencies-with-yield/>
- 36. Nested dependencies - <https://fastapi.tiangolo.com/tutorial/dependencies/sub-dependencies/>
- 38. OAuth2 password request - <https://fastapi.tiangolo.com/tutorial/security/simple-oauth2/>
- 40. OPTIONS request - <https://fastapi.tiangolo.com/tutorial/cors/>
- 42. Path validation - <https://fastapi.tiangolo.com/tutorial/path-params-numeric-validations/>
- 44. Pydantic integration - <https://fastapi.tiangolo.com/tutorial/body/>
- 46. Query parameter - <https://fastapi.tiangolo.com/tutorial/query-params/>
- 48. Response headers - <https://fastapi.tiangolo.com/advanced/response-headers/>
- 50. Router dependencies - <https://fastapi.tiangolo.com/tutorial/bigger-applications/>
- 52. Run with FastAPI CLI - <https://fastapi.tiangolo.com/fastapi-cli/>
- 54. Security scopes - <https://fastapi.tiangolo.com/advanced/security/oauth2-scopes/>
- 56. Set cookies - <https://fastapi.tiangolo.com/advanced/response-cookies/>
- 58. SQLAlchemy sync session - https://docs.sqlalchemy.org/en/20/orm/session_basics.html
- 60. Status code - <https://fastapi.tiangolo.com/tutorial/response-status-code/>
- 62. TestClient - <https://fastapi.tiangolo.com/tutorial/testing/>
- 64. Trusted host middleware - <https://fastapi.tiangolo.com/advanced/middleware/>
- 66. WebSocket endpoint - <https://fastapi.tiangolo.com/advanced/websockets/>

Vendor documentation & tools

- 68. API versioning (URL) - <https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>

Articles, guides, & named-author resources

- 69. ARQ for async tasks - <https://arq-docs.helpmanual.io/>
- 71. Class-based views - <https://fastapi-utils.davidmontague.xyz/user-guide/class-based-views/>
- 73. Unicorn + Uvicorn - <https://unicorn.dev/deployment/>
- 75. Multiple background tasks - <https://starlette.dev/background/>
- 77. Redis caching - <https://redis.io/docs/latest/integrate/fastapi/>
- 79. Test with pytest fixtures - <https://docs.pytest.org/en/stable/how-to/fixtures.html>
- 70. Celery integration - <https://docs.celeryq.dev/>
- 72. GraphQL integration - <https://strawberry.rocks/docs/integrations/fastapi>
- 74. MessagePack serialization - <https://github.com/florimondmanca/msgpack-asgi>
- 76. Rate limiting - <https://pypi.org/project/slowapi/>
- 78. Test file upload - <https://www.python-httpx.org/quickstart/#sending-multipart-file-uploads>